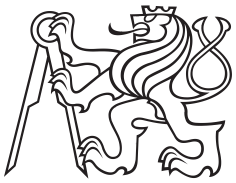
Master Thesis



Czech Technical University in Prague

F3

Faculty of Electrical Engineering
Department of Computer Science

Neurogenesis of Neural Networks for 3d Nesting

Filip Špidla

Supervisor: Ing. Mgr. Ladislava Smítková Janků, Ph.D.
Field of study: Artificial Intelligence
August 2025

Acknowledgements

Děkuji ČVUT, že mi je tak dobrou *alma mater*.

Declaration

Prohlašuji, že jsem předloženou práci vypracoval samostatně, a že jsem uvedl veškerou použitou literaturu.

V Praze, 22. August 2025

Abstract

This thesis proposes a learning-guided system for 3D nesting and transport of convex items that optimizes utilization under non-overlap, transport and container-bound constraints. It aims to explore use of neurogenesis in order to appraise its viability for this class of problems.

It proposes state exploration using heuristics to enumerate feasible loading plans combined with neural scoring models and architectures to rank candidate solutions. Model architectures are evolved via neurogenesis.

Items, containers, and candidates are represented with compact local and global features capturing size, capacity usage, and transport attributes. We survey the state of the art and build a reproducible benchmark. Evaluation reports utilization, bin count, cost, and runtime.

Keywords: nesting, neurogenesis, artificial intelligence, bin packing

Supervisor: Ing. Mgr. Ladislava Smítková Janků, Ph.D.

Abstrakt

Tato práce navrhuje systém pro 3D nesting konvexních objektů řízený učením, který optimalizuje využití prostoru při dodržení vymezení jako je nepřekrývání, přepravních požadavků a atributy kontejneru. Cílem je prozkoumat použití neurogeneze a posoudit její vhodnost pro tuto třídu úloh.

Navrhuje prozkoumávání stavového prostoru pomocí heuristik, které enumerují proveditelné plány nakládky, v kombinaci s neuronovými skórovacími modely a architekturami pro řazení kandidátních řešení. Architektury modelů jsou vyvíjeny evolučně pomocí neurogeneze.

Položky, kontejnery a kandidáti jsou reprezentováni kompaktními lokálními i globálními vlastnostmi zachycujícími rozměry, využití kapacity a přepravní atributy. Posuzuje současné state-of-the-art řešení této třídy problému a navrhuje reprodukovatelný benchmark. Vyhodnocení bude uvádět míru využití, počet binů, náklady a dobu běhu.

Klíčová slova: polohování, neurogeneze, umělá inteligence, bin packing

Překlad názvu: Neurogeneze neuronových sítí pro 3D nesting

Contents

Figures

Tables

Chapter 1

Introduction

The 3d bin-packing problem is a common optimization challenge in which items must be placed into a limited number of bins while maximizing space utilization. As the problem is NP-hard, computing an optimal solution with deterministic algorithms quickly becomes infeasible as the number of items grows.

Classical heuristic methods can be fast and effective in specific regimes, but their decision rules are difficult to tune and often fail to transfer when constraint sets or item distributions change.

This thesis explores the idea that neural networks, designed through neuro-genesis—the evolutionary construction and tuning of neural architectures-can help address these challenges. The proposed approach combines state-of-the-art geometric heuristics to generate feasible placement candidates, while an evolved neural network evaluates these candidates and selects the next action. Because the underlying nesting function is highly complex and varies with different constraint combinations, evolutionary search provides a promising way to discover neural architectures that can adapt to this variability.

Chapter 2

Problem analysis

This chapter serves as an introduction to concepts this work addresses and outlines scope of this work.

It formally defines the 3D bin packing problem. We then introduce neurogenesis as the core methodology for evolving neural network architectures in this discrete optimization setting.

In addition, the chapter provides an overview of reinforcement learning, and deep reinforcement learning in particular, as these components forms important theoretical foundation for the method developed in this work.

2.1 Problem definition

2.1.1 Bin packing

Cutting and packing problems can be broadly summarized with following description.
Given are two sets of elements, namely

- a set of large objects (input, supply) and

- a set of small items (output, demand),

which are defined in some number of geometric dimensions. Select some or all small items, group them into one or more subsets and assign each of the resulting subsets to one of the large objects such that the geometric condition holds, i.e. the small items of each subset have to be laid out on the corresponding large object such that

- all small items of the subset lie entirely within the large object and
- the small items do not overlap,

and a given (single-dimensional or multi-dimensional) objective function is optimised.[WHS07]

Definition 2.1 (3D bin packing)**.** Let $I = \{1, \dots, n\}$ be items with sizes $(w_i, h_i, d_i) \in \mathbb{R}^3_{>0}$ and per-item sets of allowed orthogonal rotations $R_i \subseteq S_3$. Bins are identical axis-aligned boxes of size $(W, H, D) \in \mathbb{R}^3_{>0}$.

A *packing* assigns each item i a bin index $k \in \{1, \dots, K\}$, an orientation $r_i \in R_i$, and a position $\mathbf{x}_i = (x_i, y_i, z_i) \in \mathbb{R}^3_{\geq 0}$ such that:

(in-bounds) $0 \leq x_i \leq W - w_i^{r_i}, \;\; 0 \leq y_i \leq H - h_i^{r_i}, \;\; 0 \leq z_i \leq D - d_i^{r_i},$

(non-overlap) $\forall i \neq j$ in the same bin k : $(x_i{+}w_i^{r_i} \leq x_j) \;\vee\; (x_j{+}w_j^{r_j} \leq x_i) \;\vee$
$(y_i{+}h_i^{r_i} \leq y_j) \;\vee\; (y_j{+}h_j^{r_j} \leq y_i) \;\vee (z_i{+}d_i^{r_i} \leq z_j) \;\vee\; (z_j{+}d_j^{r_j} \leq z_i),$

where $(w_i^{r_i}, h_i^{r_i}, d_i^{r_i})$ denotes the dimensions of item i permuted by r_i.

Objectives. Either (i) *min-bins*: pack all items using the minimum number K of bins; or (ii) *max-utilization*: given a bin budget K, pack a subset $J \subseteq I$ to maximize $\sum_{i \in J} w_i^{r_i} h_i^{r_i} d_i^{r_i}$ (equivalently, utilization per bin).

Additional constraints. Let $\mathcal{C}$ denote application-specific constraints (e.g., weight limits, stability/center-of-mass, accessibility/LIFO, orientation allowances, compatibility/segregation). A packing is *$\mathcal{C}$-feasible* if it satisfies the geometric conditions above and all $c \in \mathcal{C}$.

We target 3D bin packing with multiple bins: given a set of rectangular boxes (convex items) with orthogonal rotations, pack them efficiently into minimum number of identical containers[WHS07]

2.1.2 Constraints

Cutting and packing problems are further defined by their constraints. Beyond the basic geometric feasibility (in-bounds, non-overlap), the literature further mentions

Orientation item-specific allowed orientations, fragile handling, stackability classes[dNAJ21]

Global capacity / weight per-bin weight limits, sometimes combined with volumetric limits[dNAJ21]

Stacking / load-bearing limits on vertical support to avoid damaging items, may depend on item type or allowable stack height.[GTMP22]

Stability & center of mass static equilibrium, and even dynamic stability formulations to avoid tip-over during handling or motion[GOGL16]

Accessibility LIFO or door-side accessibility so earlier-stop items are not blocked. Some works use soft penalization rather than hard constraints[BTC23]

2.2 Neurogenesis

In the context of this work, neurogenesis refers to the evolution of neural network architectures, drawing an analogy from biological neurogenesis—the process by which new neurons are formed in the brain. **Ding et al. (2010)** forms the basis of our approach to neurogenesis.

More specifically, we aim to study application of automated design of neural network architectures through evolutionary computation. Unlike traditional deep learning, where architectures are manually designed and only weights are learned, neurogenesis treats both topology and parameters as evolvable components [DXSZ10].

For the 3D bin packing problem, neurogenesis offers several advantages that make it particularly suitable:

No gradient requirement. The discrete, sequential nature of placement decisions means we cannot backpropagate gradients through the packing

process. Evolutionary methods circumvent this by using outcome-based fitness evaluation rather than gradient descent.

Multi-objective optimization. Bin packing involves multiple competing objectives (utilization, bin count, constraint satisfaction, runtime). Genetic algorithms naturally handle multi-objective fitness through Pareto-based selection or weighted scalarization [EMH19].

Architecture discovery. Different constraint combinations may benefit from different neural inductive biases. For example, spatial constraints might favor convolutional processing of heightmaps, while compatibility constraints might benefit from graph neural network components. Rather than manually testing all architectural combinations, evolutionary search can discover which components work best for specific problem variants.

Constraint-specific adaptation. Since real-world packing problems vary widely in their constraint sets (weight limits, stability requirements, accessibility constraints), a single hand-designed architecture is unlikely to be optimal across all variants. Neurogenesis allows the architecture itself to adapt to the specific constraint profile of each problem class.

The detailed methodology of neurogenesis—including genome encoding schemes, evolutionary operators, and fitness design—is reviewed in Section 3.3 as part of the state-of-the-art survey.

2.3 Reinforcement learning

Reinforcement learning (RL) is a framework for sequential decision-making in which an agent interacts with an environment in discrete time steps, observes its state, selects actions, and receives scalar rewards. The agent's objective is to learn a policy that maximizes the expected cumulative reward over time.[SB18] Formally, this interaction is often modelled as a Markov decision process (MDP) with state space $\mathcal{S}$, action space $\mathcal{A}$, transition kernel $P(s' \mid s, a)$, and reward function $r(s, a)$.

At each time step t, the agent observes a state $s_t \in \mathcal{S}$, chooses an action $a_t \in \mathcal{A}$, and then receives a reward $r_t = r(s_t, a_t)$ while the environment transitions to a new state $s_{t+1} \sim P(\cdot \mid s_t, a_t)$. The goal is to find a policy

$\pi(a \mid s)$ that maximizes the expected return

$$\mathbb{E}\left[\sum_{t=0}^{T} \gamma^t r_t\right],$$

where $\gamma \in [0, 1)$ is a discount factor and T is the episode horizon.[SB18]

2.3.1 Learning for 3D nesting

For 3D nesting with realistic industrial constraints, large labeled datasets of "optimal" or expert packings are scarce. Existing datasets and benchmarks typically provide item sets and container specifications, but not sequences of placements or even universally accepted best layouts. Moreover, even if such datasets were available, supervised learning would be limited to at best reproducing the behavior implicit in the labels, rather than exploring policies that may improve upon existing heuristics.

Objective of this work is not merely to imitate current bin-packing heuristics, but to search for the best nesting strategies given constraints. Reinforcement learning is particularly well-suited to this setting for several reasons:

2.3.2 Deep Reinforcement Learning

Classical reinforcement learning algorithms such as tabular Q-learning and policy iteration maintain explicit value tables with one entry per state-action pair. This approach becomes computationally intractable when the state space exceeds approximately 10^6 states due to memory constraints and the sample complexity required to visit each state sufficiently often. For example, a 10×10 grayscale image has $256^{100} \approx 10^{240}$ possible states—far beyond the reach of tabular methods.

Deep reinforcement learning addresses this scalability barrier by using neural networks as function approximators: instead of storing $Q(s, a)$ in a table, we represent it as $Q(s, a; \theta)$ where θ are the parameters of a deep neural network. This enables generalization across similar states, dramatically improving practical sample efficiency compared to tabular methods. Empirical results demonstrate that neural function approximation can successfully scale to problems with state spaces orders of magnitude larger than feasible for tabular approaches.[MKS+13]

- **Learning from interaction instead of labels.** The packing environment can be simulated, so an agent can generate its own experience by repeatedly constructing packings and observing rewards that encode utilization, constraint violations, and runtime. No pre-labeled dataset of "correct" actions is required.[SB18]
- **Direct optimization of task-specific objectives.** The reward function can combine multiple criteria (e.g., fill ratio, stability, accessibility penalties), allowing the agent to directly optimize the same objectives that define solution quality in practice.[MSIB20]
- **Potential to outperform hand-crafted heuristics.** Since the agent is free to explore the space of decision policies, it is not constrained to mimic any single heuristic and should theoretically be able to beat the existing heuristics.

2.4 Neural Scoring Models

While heuristics can efficiently enumerate feasible candidate placements using extreme points or empty maximal spaces, the core difficulty in constrained 3D nesting is **selecting among many feasible candidates** under heterogeneous objectives. Classical approaches rely on hand-crafted scoring rules that prioritize specific metrics (e.g., minimize residual void, maximize support area), but these rules are brittle and fail to generalize when constraint sets or item distributions change.

The role of learned scoring. A neural scoring model replaces fixed tie-breakers with a parameterized function that learns to rank placement candidates from outcome-based feedback. At each packing step, the model evaluates candidates using:

- **Item features**: dimensions, weight, orientation constraints, fragility class
- **Placement features**: position, residual void, support surface, clearance
- **Global state**: current utilization, bin capacity, item count

The network outputs a scalar score; the highest-scoring feasible placement is selected. This decomposition—deterministic candidate generation followed by

learned selection—preserves feasibility guarantees while enabling adaptation to complex, multi-objective fitness landscapes.

Why neural networks? The discrete, non-differentiable nature of placement decisions precludes gradient-based optimization through the packing process. Neural scoring models accommodate this by:

1. Operating as **black-box evaluators** within a constructive heuristic
2. Accepting compact, engineered feature vectors rather than raw geometry
3. Enabling outcome-based training via evolutionary search (Section 3.3)

This formulation treats the packing pipeline as a Markov decision process where the policy (scoring model) is evolved rather than trained by gradients, fitting naturally into the neurogenesis framework while maintaining computational tractability during inference.

2.5 Problem Characteristics

Given this analysis, we arrive at several challenges and particularities that will have to be addressed by proposed solution:

- **Sparse Reward Structure**: RL rewards are naturally sparse in sequential packing - only terminal/infrequent feedback
- **Large and Fragmented Search Space**: Combinatorial explosion of positions, EMS fragmentation, exponential branching
- **Architectural Compatibility Requirements**: Neurogenesis operates within structured ANN architectures - genome must encode valid networks
- **Computational Constraints**: Fast evaluation required for larger instances and GA population assessments
- **Absence of Labeled Data**: Fast evaluation required for larger instances and GA population assessments
- **Geometric Invariances**: Input equivariance properties - rotational symmetries, position invariances

2.6 Scope of This Work

This work combines neural architecture search with deep reinforcement learning for the 3D bin packing problem. We aim to demonstrate that genetic algorithms can automatically discover neural network architectures suited to constraint-specific packing scenarios without manual hyperparameter tuning.

2.6.1 Problem Scope

We address offline 3D bin packing where all items are known in advance. Items are rectangular boxes with axis-aligned orientations, gravity, multiple bins and constraints may be present.

We do *not* address online/dynamic packing (items arriving in real-time), vehicle routing integration, non-convex items or arbitrary 3D rotations.

2.6.2 Technical Description

The system integrates three key components:

Bin packing. Thesis needs to create an effective bin packing environment for 3D cuboid geometry enables gravity simulation, spatial feature extraction and supports chosen constraints. Upon it a heuristic that generates geometrically feasible placement positions.

Neural Scoring Model. Neural network evaluates candidate placements. The architecture (layer depth, hidden dimensions, attention type, activation functions, dropout) is determined by evolutionary search rather than manual design.

Genetic Architecture Search. A genetic algorithm evolves neural network architectures in order to facilitate improved learning capabilities. Neuroge-

nesis operators apply over multiple generations. Fitness combines packing utilization, bin count, and model complexity.

2.6.3 Evaluation Strategy

We evaluate on available public benchmark datasets. The goal is to demonstrate that:

1. Evolved architectures outperform random architectural choices
2. The system can learn effective placement strategies under multiple constraint types
3. Genetic search discovers problem-appropriate inductive biases (e.g., convolutional spatial reasoning, attention over actions)

Chapter 3

State-of-the-art solutions

After defining the problem of three-dimensional bin packing and motivating the need for an efficient, adaptive optimization solution, this chapter reviews the current state-of-the-art in related methods. The goal is to summarize the existing approaches and appraise them based on problem characteristics in section 2.5. We will further highlight the research gaps that need to be patched up in chapter 4.

This chapter is divided into Three main sections. The first section explores nesting, focusing on heuristic methods developed for the 3D bin packing problem. The second section focuses on neural networks. Third section is about neurogenesis and related methods of efficient neural architecture search. Many papers intersect in some parts or use concepts talked about in other sections, they are assigned into sections based on their primary contribution to this work. Lastly, we shall mention which problem of this task in particular are currently lacking in state-of-the-art and will need to be figured out on its own in this work.

3.1 Nesting

This section categorizes solution strategies for the class of nesting problems. Individual proposed approaches can be broadly split by several characteristics which play a crucial role in determining how algorithm may approach nesting. They provide a handy overview when evaluating an approach's usefulness in

particular scenarios, so I shall list several of them at the beginning of this section.
Rest of the section outlines current state-of-the-art algorithms for class of nesting based problems, along with papers focusing on certain important components such as candidate placements in particular.

Information model (offline vs. online). In the offline setting, the entire item set is known, enabling pre-sorting and deeper look-ahead; online packing processes items as they arrive and must make irrevocable decisions. State-of-the-art strategies in both regimes share a constructive skeleton (candidate generation → selection) but differ in how they balance exploration, robustness, and speed; online variants typically adopt myopic but well-regularized scoring rules and occasional re-packing windows [ARCO22].

Space representation (rasterization vs. geometry-first). Raster/voxel models offer simple, parallelizable collision checks and convenient stability features at the cost of resolution dependence. Geometry-first models rely on exact or approximate intersection tests for convex bodies, oriented bounding boxes, separating-axis or GJK/EPA checks, and precomputed structures (no-fit relations in 2D; extreme points or empty maximal spaces in 3D). Hybrid designs are common: geometry for candidate generation and feasibility, rasterized descriptors for local voids/support [ARCO22].

Constructive heuristics. Constructive backbones couple item ordering (e.g., by size, shape complexity, or "hardness") with candidate evaluation rules (minimize residual void, maximize support, minimize overhang, respect separations and orientation sets). These rules deliver fast feasible layouts and provide consistent starting points for improvement phases [ARCO22].

Neural scoring and learned proposals. Neural components most often rerank EP/EMS candidates instead of proposing arbitrary poses. Feature sets typically mix void/waste metrics, clearances, support/contact estimates, and simple surrogates for load distribution or access. Lightweight models (MLPs, attention over placed items) can replace hand-crafted tie-breakers without changing the surrounding heuristic or matheuristic scaffold [ARCO22].

Local improvement. After a feasible placement, local refinements (swap, rotate within allowed sets, reinsert, neighborhood re-pack) and compaction steps reduce fragmentation. Physics-inspired or projection-based moves

gently compress the layout; when overlaps appear, constraints are restored by expanding or re-projecting to the feasible set. Such micro-optimizations target the highest-void regions and are effective under strict time budgets [ARCO22].

Metaheuristics, matheuristics. GRASP, VNS, tabu, and LNS frameworks explore item orderings and partial re-packs; matheuristics embed exact subproblems (e.g., orientation/placement pricing, stability/separation checks) to sharpen feasibility and improve bounds. This hybridization remains the most common route to high fill ratios with realistic industrial constraints [ARCO22].

3.1.1 Candidate placement heuristics

Modern 3D solvers avoid global pose search by enumerating high-quality placements at the evolving packing frontier. Two families dominate: (i) *extreme points*, induced by faces/edges/vertices of the container and already placed items, and (ii) *empty maximal spaces*, maintained as items are inserted. Both admit quick filters for container bounds, non-overlap, minimal support, separations, and admissible orientations [ARCO22].

Extreme points. Crainic et al. (2007)] introduces concept of extreme points and propose new extreme point based online, geometric heuristic for packing items inside 3D container. [CPT08]

Given a packing and the list `3DEPL` of Extreme Points (EPs) defined by the items already placed, Algorithm 1 computes the new EPs that must be added to the list after placing item k at position (x_k, y_k, z_k). The main idea is as follows:

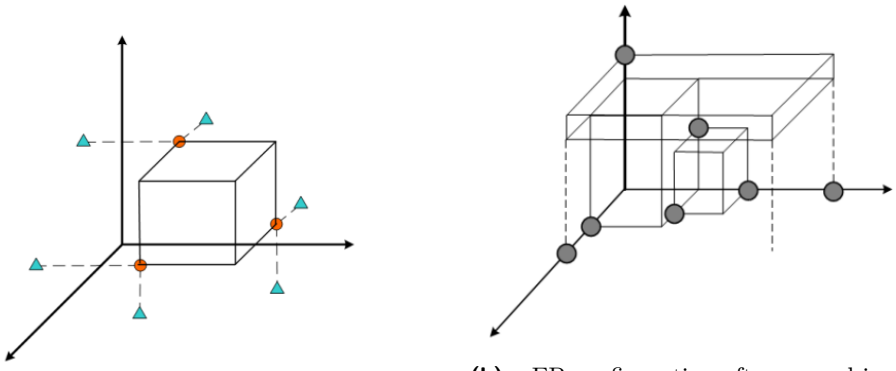
- **Empty container:** If the container is empty, item k is placed at $(0, 0, 0)$, which generates the following three EPs:

$$(w_k,\, 0,\, 0), \quad (0,\, d_k,\, 0), \quad (0,\, 0,\, h_k).$$

- **Non-empty container:** When item k is placed at (x_k, y_k, z_k), new EPs are generated by projecting:
  - the point $(x_k + w_k,\ y_k,\ z_k)$ along the Y and Z axes,

- the point $(x_k,\ y_k + d_k,\ z_k)$ along the X and Z axes,
- the point $(x_k,\ y_k,\ z_k + h_k)$ along the X and Y axes.

- **Projection rule:** Each projected point is moved until it touches either another item or the container wall in the respective direction. If more than one item is intersected along the projection direction, the algorithm chooses the nearest one.



(a) : EP projections from one insertion.

(b) : EP configuration after several insertions.

Figure 3.1: Illustration of Extreme Point generation and accumulation[ARCO22]

Algorithm 1 Generation of new Extreme Points after placing item k

Require: Current packing; list of EPs `3DEPL`; item k with size (w_k, d_k, h_k) placed at (x_k, y_k, z_k)
Ensure: Updated list `3DEPL`

if container is empty **then**
    place item k at $(0, 0, 0)$
    add EPs: $(w_k, 0, 0),\ (0, d_k, 0),\ (0, 0, h_k)$
    return `3DEPL`
end if
define candidate points:

$$p_1 = (x_k + w_k,\ y_k,\ z_k), \quad p_2 = (x_k,\ y_k + d_k,\ z_k), \quad p_3 = (x_k,\ y_k,\ z_k + h_k)$$

for each candidate point p_i **do**
    determine relevant projection axes
    project p_i along the corresponding directions
    stop projection when hitting nearest item or container wall
    add resulting EP to `3DEPL`
end for
return updated `3DEPL`

The paper introduces the *Extreme Points First Fit Decreasing* (EP–FFD) heuristic, a packing method built around the notion of *extreme points* (EPs). The heuristic first sorts the items using an externally specified ordering rule and then places them sequentially into the lowest feasible (x, y, z) position

among the currently available EPs. The authors report a time complexity of $O(n^3)$, reflecting the cost of evaluating feasibility and updating the EP set after each placement.

Although the concept of extreme points is foundational, the original work omits several mechanisms essential for practical performance. Notably, it does not describe how to *prune* or *cull* dominated or irrelevant EPs, despite illustrations in the paper suggesting that some form of culling is implicitly applied - Figure 3.1b. Without such pruning, the EP set can grow rapidly, leading to unnecessary collision checks and increased computational cost.

Subsequent research introduced important refinements. Other works generalized the EP framework beyond cuboid geometries, handling arbitrary convex shapes and integrating industrial constraints such as stackability, weight limits, and load-bearing checks.[GTMP22] **Heßler et al. (2024)** extended EP-based heuristics by incorporating efficient space subdivision for faster collision detection [HHW24].

Overall, the extreme point paradigm remains a widely used candidate–placement baseline across many state-of-the-art nesting and three-dimensional bin packing algorithms.

Maximal empty spaces. While EPs represent point locations, the EMS heuristic maintains a set of rectangular *volumes*—specifically, axis-aligned boxes representing empty regions within the container. **Parreño et al. (2008)** serves as a good introduction maximal-space representation for 3D bin packing problems. Their work outlines an algorithm which generates and maintains empty maximal spaces as boxes are placed.

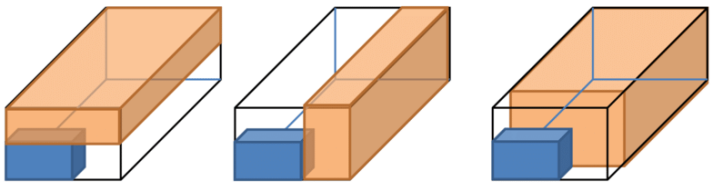


Figure 3.2: Block (blue) creates new empty maximal-spaces (orange)[LZZ14].

An empty maximal space in 3D is defined by its corner position (x, y, z) and dimensions (w, d, h), representing the largest axis-aligned rectangular void at that location. The EMS set is dynamically updated after each placement using *difference operations*: when an item is placed, each EMS that intersects the item's volume is subdivided into up to six new maximal spaces corresponding

to the six axis-aligned slices (left, right, front, back, bottom, top) remaining after the intersection. To prevent combinatorial explosion, EMS that are fully contained within other EMS are eliminated through *dominance pruning*—if space A is entirely within space B, A is discarded since any item fitting in A could also be placed in B at a potentially better position. Extracted from the paper, following algorithm describes Update of Maximal empty spaces after placing a block.[PAVTO08]

Algorithm 2 Update of maximal-spaces list L after placing a block in S^*

Require:
L – current list of empty maximal-spaces
$S^* \in L$ – maximal-space where the block B has just been packed
B – placed block (axis-aligned rectangular block of boxes)
Ensure:
Updated list L of empty maximal-spaces

Step 3.1: Replace S^* by new spaces created inside it
if B exactly fills S^* **then**
    Remove S^* from L
else
    Remove S^* from L
    Compute the empty maximal-spaces created inside S^* around B ▷ geometric decomposition of $S^* \setminus B$
    Insert all newly created maximal-spaces into L
end if

Step 3.2: Reduce other maximal-spaces intersecting B
for all $S \in L$ such that S intersects B **do**
    Compute $S \setminus B$
    if $S \setminus B = \emptyset$ **then**
        Remove S from L
    else
        Replace S by the maximal-spaces obtained from $S \setminus B$ ▷ split into one or more smaller maximal-spaces
    end if
end for

Step 3.3: Remove contained (non-maximal) spaces
for all pairs of spaces (S_i, S_j) in L, $i \neq j$ **do**
    if $S_i \subset S_j$ **then**
        Remove S_i from L
    else if $S_j \subset S_i$ **then**
        Remove S_j from L
    end if
end for
return L

Given This update mechanism, paper proposes a *greedy randomized adaptive search procedure* (GRASP), an offline, geometrical, partially randomized 3D nesting algorithm with local improvement.

At each iteration, the algorithm selects the maximal-space $S^* \in L$ that is lexicographically closest to a container corner, and anchors the next placement at that corner. For all box types fitting into S^*, the method generates feasible *blocks* (columns or layers of identical boxes) and evaluates them using either volume increase or a best-fit criterion. The best-scoring block is packed into S^*.

After placement, the list of maximal-spaces is updated: S^* is replaced by the new empty regions created around the block; any other maximal-spaces intersecting the block are reduced; and dominated spaces (those contained within others) are removed.

Fanslau and Bortfeldt (2010) present a tree search algorithm using EMS for the container loading problem with weak heterogeneity (items with similar dimensions). They demonstrate that maintaining EMS sorted by volume and applying pruning—keeping only the top-k largest spaces—significantly improves computational efficiency without greatly reducing solution quality.[FB10]

Gonçalves and Resende (2013) [GR13] proposes a biased random-key genetic algorithm for 3D bin packing (BRKGA). Their approach combines EMS-based placement with evolutionary item ordering, achieving state-of-the-art results on benchmark instances. They show that EMS-based heuristics naturally encode both spatial efficiency (preferring placements in large voids to minimize fragmentation) and stability (bottom-left placement bias ensures support from below).

Xiong et al. (2024) introduce a generalizable online 3D Bin Packing approach(GOPT), a Transformer-based deep reinforcement learning method for online 3D bin packing in real setting using robotic arm. Their approach represents free space using a compact set of rasterized EMSs extracted from heightmaps, enabling a fixed action space independent of bin size. A Packing Transformer models interactions between items and EMSs via attention, producing placement policies that generalize across bins of different dimensions.[XGP+24].

In summary, EMS-based methods provide a geometrically expressive and computationally tractable framework for representing free volume in 3D

packing. Unlike EP heuristics, which enumerate pointwise placements, EMS encodes full empty regions and therefore supports richer operations such as dominance pruning or more accurate feature extraction.

This has made EMS a central component in many high-performing solvers, from classical GRASP-based approaches to evolutionary and genetic metaheuristics, and even modern transformer-driven reinforcement learning systems. Despite methodological differences, these works highlight the same principle: maintaining an accurate, pruned set of maximal empty spaces turns the complex 3D geometry of nesting into a structured action space that can be exploited effectively by both heuristic and learning-based decision models.

3.1.2 Feature extraction

Feature extraction transforms the complex 3D geometric state of a bin-packing problem into a compact representation suitable for decision-making, whether by hand-crafted heuristics or learned neural models. The choice of features directly impacts solution quality, it is hard for learning models to understand the geometric representation with mere positions and sizes. Too sparse and critical spatial relationships are lost; too rich and computational cost becomes prohibitive.[WFH+23] This section reviews how state-of-the-art methods—both traditional and learning-based—represent packing states to guide placement decisions.

Geometric Features from Candidate Heuristics. EP and EMS heuristics subsection 3.1.1 inherently define feature spaces through their geometric primitives. Each EP or EMS encodes local spatial information like position or available volume in the case of EMS. These geometric descriptors are used directly by constructive heuristics to rank candidates. For example, the EP-FFD heuristic[CPT08] selects placements lexicographically prioritizing lower (x, y, z) coordinates, implicitly favoring compact, bottom-left arrangements. EMS-based methods[PAVTO08, GR13] often prefer larger spaces to minimize fragmentation.

Heightmap Representation. A heightmap is a 2D grid representation of the maximum occupied height at each (x, y) position in the container. Originally developed for 2D bin packing with gravity constraints, heightmaps have been adapted for 3D nesting to efficiently encode vertical occupation and support surfaces [ARCO22].

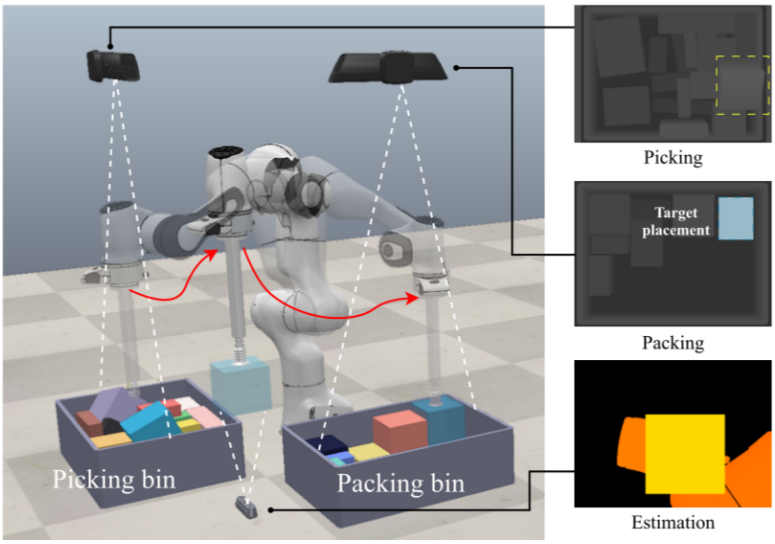


Figure 3.3: Robotic arm using raster heightmap to comprehend bin packing environment[XGP+24].

Construction and Usage. Given a container of dimensions $W \times D \times H$, a heightmap discretizes the base (x, y) plane into a grid of resolution $r_x \times r_y$. Each cell (i, j) stores the maximum z-coordinate of any item occupying that horizontal position:

$$h[i, j] = \max \left\{ z + d_z^k \mid \text{item } k \text{ occupies cell } (i, j) \right\} \tag{3.1}$$

where d_z^k is the height of item k in its current orientation. If no item occupies cell (i, j), then $h[i, j] = 0$.

Key Advantages:

1. **Efficient support detection**: An item placed at (x, y, z) is supported if its footprint aligns with heightmap values $h[i, j] \approx z$ within tolerance [XGP+24]
2. **Void identification**: Large flat regions in the heightmap indicate stable placement zones [ARCO22]
3. **Gravity simulation**: Items can be "dropped" onto the heightmap by setting $z = \max_{(i,j) \in \text{footprint}} h[i, j]$ [XGP+24]
4. **Fixed representation size**: Unlike EP/EMS lists that grow with

the number of placements [CPT08, PAVTO08], heightmaps maintain constant $O(x \times y)$ memory

Limitations:

- **Resolution trade-off**: Fine grids capture detail but increase computational cost; coarse grids may miss small voids or create artificial overlaps [ARCO22]
- **Loss of 3D structure**: Overhangs and internal cavities cannot be represented; only the topmost surface is recorded
- **Discretization artifacts**: Continuous item positions must be mapped to discrete grid cells, potentially introducing small gaps or overlaps

Handcrafted Features in Traditional Methods. Classical heuristics and metaheuristics rely on explicitly engineered features to evaluate placement quality. These features typically decompose into three categories:

Item-Level Features. Describe properties of the item to be placed, inherently restricting item's available placement are relevant when determining available placements and planning ahead placement strategies

- **Dimensions** (w, h, d) in current orientation
- **Volume** and **aspect ratios** (e.g., w/h, capturing item "shape")
- **Task-specific features** fragility, stacking constraints, set of allowed rotations

Placement-Level Features. Quantify the quality of placing an item at a specific candidate position:

- **Residual void**: Volume of empty space created around the item after placement
$$V_{\text{void}} = V_{\text{EMS}} - V_{\text{item}} \tag{3.2}$$
Minimizing void reduces fragmentation [PAVTO08].

- **Support area**: Footprint overlap with items below or container floor. Higher support improves stability [RSO18].
- **Wall contact**: Number of container faces the item touches. Maximizing contact tends to consolidate items [CPT08].
- **Clearance**: Distance between `candidate columns` of items/container boundaries[MZF+25].
- **Height penalty**: Vertical position z; lower placements typically preferred to maintain a compact profile.[PAVTO08]

Global State Features. Capture the overall packing configuration:

- **Current utilization**: $\sum_{k \in \text{placed}} V_k / V_{\text{container}}$
- **Number of items remaining** to be packed
- **Bin count** (for multi-bin problems): penalizes opening new bins
- **Center-of-mass deviation**: Distance from container centroid, used for stability constraints [RSO18]
- **Fragmentation metrics**: Number of disjoint empty regions or variance in heightmap values[ZZOL12]

These features can be further combined into scoring functions via weighted sums or lexicographic ordering. For example, **Gonçalves and Resende (2013)** rank EMS candidates by:

$$\text{score}(\text{EMS}, \text{item}) = \lambda_1 \cdot V_{\text{EMS}} - \lambda_2 \cdot V_{\text{void}} + \lambda_3 \cdot \text{support}(\text{item}, \text{EMS}) \quad (3.3)$$

where weights λ_i are hand-tuned or evolved via genetic algorithms.[GR13]

Neural networks and features. When packing state is represented as a 2D heightmap or 3D voxel grid, **convolutional neural networks (CNNs)** extract spatial features hierarchically:

- **Low-level**: Edge detectors identify container boundaries and item interfaces
- **Mid-level**: Texture patterns capture repeating structures (e.g., uniform stacking)

- **High-level**: Spatial pooling aggregates information across the entire container, detecting global fragmentation or imbalance

Xiong et al. (2024)[XGP+24] apply 2D convolutions to heightmaps, followed by spatial pooling to produce a fixed-size feature vector. This vector is concatenated with item embeddings and processed by a Transformer to predict placement scores.

Attention-Based Context Aggregation. Beyond simple MLPs, recent work incorporates **attention mechanisms** [XGP+24] to model interactions between:

- **Items**: Which item dimensions are complementary for tight packing?
- **Candidates**: Which placements leave the most favorable residual space for future items?
- **Placed items**: Does the current configuration block high-value placements?

Transformer layers in GOPT - **Xiong et al. (2024)** process the EMS and item embeddings using the standard scaled dot-product attention mechanism [VSP+17]. The Packing Transformer applies both self-attention (to model relations among EMSs) and bi-directional cross-attention (to fuse EMS and item features), where EMS embeddings and item embeddings alternately serve as queries, keys, and values [XGP+24]. This design enables the policy network to extract spatial correlations between the current item and available empty maximal spaces.

Comparison: Handcrafted vs. Learned Features. Hybrid approaches—using geometric features as input to lightweight neural scoring models—offer a promising middle ground, combining the efficiency and interpretability of handcrafted features with the adaptability of learned decision-making.

Summary. Feature extraction in 3D bin packing serves to compress the complex spatial state into representations suitable for placement evaluation. Traditional methods tend to rely on interpretable geometric features (void volume, support area, wall contact) combined via weighted scoring functions or lexicographic rules.

Aspect	Handcrafted Features	Learned Features (Neural)
Interpretability	High: each feature has clear geometric meaning	Low: learned representations are opaque
Generalization	Brittle: fixed rules may not transfer across constraint sets	Strong: networks adapt to different distributions via training
Computational Cost	Low: simple arithmetic on geometric primitives	Moderate to High: forward passes through deep networks
Data Requirements	None: features are manually designed	High: requires large datasets or extensive simulation
Flexibility	Limited: adding new constraints requires redesigning features	High: retraining with new reward function adapts automatically

Table 3.1: Comparison of handcrafted and learned feature extraction approaches for 3D bin packing.

Modern learning-based approaches embed items, candidates, and global state into neural networks–often augmented with heightmaps for spatial reasoning or attention mechanisms for relational context. Heightmaps in particular bridge the gap between discrete geometric heuristics and continuous visual encodings, enabling convolutional architectures to learn spatial patterns that generalize across problem instances.

3.2 Neural Networks

This section surveys neural network architectures relevant to discrete optimization and combinatorial problems. While the application to 3D nesting will be addressed in later chapters, this section establishes the theoretical foundations of modern deep learning architectures that have shown promise in learning complex decision policies.

3.2.1 Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are specialized neural architectures designed to process grid-structured data, originally developed for computer vision tasks [LBBH98]. Their key innovation lies in the use of convolution operations that preserve spatial structure while learning hierarchical feature representations. The success of deep CNNs was decisively demonstrated by AlexNet [KSH12], which achieved breakthrough performance on large-scale image recognition tasks.

Architecture and Components

A CNN consists of several types of layers arranged in a feed-forward manner:

Convolutional Layers. The core building block applies learnable filters (kernels) across the input through discrete convolution. For a 2D input $\mathbf{X} \in \mathbb{R}^{H \times W}$ and a kernel $\mathbf{K} \in \mathbb{R}^{k \times k}$, the convolution operation at position (i, j) is:

$$(\mathbf{X} * \mathbf{K})[i, j] = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} \mathbf{X}[i + m, j + n] \cdot \mathbf{K}[m, n] \tag{3.4}$$

Each convolutional layer typically applies multiple filters to learn different features, producing a set of feature maps.

Activation Functions. Non-linear activations are applied element-wise after convolution. Common choices include ReLU (Rectified Linear Unit) [DSC22]: $f(x) = \max(0, x)$, which introduces non-linearity while remaining computationally efficient, and variants such as Leaky ReLU and GELU that address specific limitations.

Pooling Layers. These layers reduce spatial dimensionality through downsampling operations. Max pooling selects the maximum value within each local region:

$$\text{MaxPool}(\mathbf{X})[i, j] = \max_{m,n \in \mathcal{N}_{i,j}} \mathbf{X}[m, n] \tag{3.5}$$

where $\mathcal{N}_{i,j}$ denotes the pooling window centered at (i, j). Average pooling computes the mean instead, providing a smoother downsampling alternative.

Fully Connected Layers. Terminal layers flatten the feature maps and apply standard dense connections to produce final outputs.

Key Properties

CNNs exhibit several properties that make them effective for spatial data:

Parameter Sharing. The same kernel weights are applied across all spatial locations, drastically reducing the number of parameters compared to fully connected networks. For a $H \times W$ input with $k \times k$ kernels, a convolutional layer uses k^2 parameters per filter, whereas a fully connected layer would require $H \cdot W$ parameters per output unit.

Translation Equivariance. Convolution operations are equivariant to translation: shifting the input results in an equivalent shift in the output. Formally, if T_δ denotes a translation operator, then:

$$\text{Conv}(T_\delta(\mathbf{X})) = T_\delta(\text{Conv}(\mathbf{X})) \tag{3.6}$$

This property allows CNNs to detect features regardless of their position in the input.

Local Receptive Fields. Each unit in a convolutional layer connects only to a local region of the input, enabling efficient processing and hierarchical feature extraction. Deeper layers implicitly have larger receptive fields, capturing increasingly global information.

Applications to Optimization Problems

While originally designed for image processing, CNNs have been adapted to combinatorial optimization through creative input representations. For problems with spatial structure, such as the Traveling Salesman Problem on Euclidean coordinates or grid-based puzzles, CNNs can process 2D or 3D grids representing problem states.

In the context of packing and nesting, heightmaps paragraph 3.1.2 provide a natural 2D representation amenable to convolutional processing [XGP+24].

The CNN can learn to detect spatial patterns such as cavities, support surfaces, and fragmentation that are relevant for placement quality assessment.

3.2.2 Recurrent Neural Networks

Recurrent Neural Networks (RNNs) [Elm90] extend feedforward architectures to process sequential data by maintaining hidden state that captures information from previous time steps. This makes them well-suited for tasks involving temporal dependencies or variable-length sequences.

Architecture

An RNN processes a sequence $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T$ by maintaining a hidden state $\mathbf{h}_t$ that is updated at each time step:

$$\mathbf{h}_t = f(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h) \tag{3.7}$$
$$\mathbf{y}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y \tag{3.8}$$

where f is a non-linear activation function (typically tanh or ReLU), $\mathbf{W}_{hh}$, $\mathbf{W}_{xh}$, and $\mathbf{W}_{hy}$ are learnable weight matrices, and $\mathbf{b}_h$, $\mathbf{b}_y$ are bias vectors.

Long Short-Term Memory (LSTM)

Standard RNNs suffer from the vanishing gradient problem when learning long-range dependencies. LSTMs address this through a gating mechanism that controls information flow. An LSTM cell maintains two state vectors: the hidden state $\mathbf{h}_t$ and the cell state $\mathbf{c}_t$, updated through three gates:[GSC00]

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad \text{(forget gate)} \tag{3.9}$$
$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad \text{(input gate)} \tag{3.10}$$
$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad \text{(output gate)} \tag{3.11}$$
$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) \quad \text{(candidate values)} \tag{3.12}$$
$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \tag{3.13}$$
$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \tag{3.14}$$

where σ denotes the sigmoid function and $\odot$ represents element-wise multiplication.

Gated Recurrent Unit (GRU)

GRUs [CvMG+14] simplify the LSTM architecture by combining the forget and input gates into a single update gate, and merging the cell state with the hidden state:

$$\mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t]) \quad \text{(reset gate)} \tag{3.15}$$
$$\mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t]) \quad \text{(update gate)} \tag{3.16}$$
$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t]) \tag{3.17}$$
$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \tag{3.18}$$

GRUs often achieve comparable performance to LSTMs with fewer parameters and faster training.

Application to Sequential Decision Making

RNNs are natural candidates for reinforcement learning problems where decisions must be made sequentially and past actions influence future states. In constructive optimization heuristics, where items are placed one at a time, RNNs can model the sequential dependency between placement decisions and learn to account for how current choices affect future options.

3.2.3 Graph Neural Networks

Graph Neural Networks (GNNs) [SGT+09] extend deep learning to graph-structured data by defining convolution-like operations on arbitrary graph topologies. Unlike CNNs, which operate on regular grids, GNNs can process irregular structures such as social networks, molecules, or combinatorial problem instances.

Graph Representation

A graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ consists of vertices (nodes) $\mathcal{V} = \{v_1, \ldots, v_n\}$ and edges $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$. Each node v_i has a feature vector $\mathbf{h}_i^{(0)} \in \mathbb{R}^d$, and optionally, edges may have features $\mathbf{e}_{ij}$.

Message Passing Framework

Most GNN architectures follow a message-passing paradigm [GSR+17], where node representations are iteratively updated by aggregating information from neighboring nodes. At layer ℓ, the update for node i is:

$$\mathbf{m}_i^{(\ell)} = \text{AGGREGATE}^{(\ell)}\left(\{\mathbf{h}_j^{(\ell-1)} : j \in \mathcal{N}(i)\}\right) \tag{3.19}$$

$$\mathbf{h}_i^{(\ell)} = \text{UPDATE}^{(\ell)}\left(\mathbf{h}_i^{(\ell-1)}, \mathbf{m}_i^{(\ell)}\right) \tag{3.20}$$

where $\mathcal{N}(i)$ denotes the neighbors of node i, AGGREGATE is a permutation-invariant function (sum, mean, max), and UPDATE is typically a feedforward neural network.

Graph Convolutional Networks (GCN)

Kipf and Welling [KW17] propose a simplified convolution operator based on spectral graph theory. For a graph with adjacency matrix $\mathbf{A}$ and node features $\mathbf{H}^{(\ell)}$, the GCN layer computes:

$$\mathbf{H}^{(\ell+1)} = \sigma\left(\tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}\mathbf{H}^{(\ell)}\mathbf{W}^{(\ell)}\right) \tag{3.21}$$

where $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ is the adjacency matrix with added self-loops, $\tilde{\mathbf{D}}$ is the degree matrix of $\tilde{\mathbf{A}}$, and $\mathbf{W}^{(\ell)}$ is a learnable weight matrix.

Graph Attention Networks (GAT)

GATs [VCC+18] introduce an attention mechanism to weight the importance of different neighbors. The attention coefficient α_{ij} between nodes i and j is computed as:

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \| \mathbf{W}\mathbf{h}_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \| \mathbf{W}\mathbf{h}_k]))} \tag{3.22}$$

where $\|$ denotes concatenation and $\mathbf{a}$ is a learnable attention vector. Node features are then updated as:

$$\mathbf{h}'_i = \sigma\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}\mathbf{W}\mathbf{h}_j\right) \tag{3.23}$$

Applications to Combinatorial Optimization

GNNs have shown strong performance on graph-based combinatorial problems such as the Traveling Salesman Problem, Maximum Cut, and Graph Coloring. Many optimization problems admit natural graph representations: in bin packing, items can be nodes with edges representing compatibility or spatial relationships; placement candidates can form a graph where edges encode geometric constraints.

3.2.4 Deep Reinforcement Learning and Deep Q-Networks

Deep Reinforcement Learning combines neural function approximation with reinforcement learning algorithms, enabling agents to learn policies directly from high-dimensional observations. This section focuses on value-based methods, particularly Deep Q-Networks and their variants.

Q-Learning and the Bellman Equation

Q-learning is a model-free RL algorithm that learns an action-value function $Q(s, a)$ representing the expected return from taking action a in state s and following an optimal policy thereafter. The optimal Q-function satisfies the Bellman optimality equation:

$$Q^*(s, a) = \mathbb{E}_{s'} \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right] \tag{3.24}$$

where r is the immediate reward, $\gamma \in [0, 1)$ is the discount factor, and s' is the next state.

Tabular Q-learning updates estimates using temporal-difference (TD) learning:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \tag{3.25}$$

where α is the learning rate.

Deep Q-Networks (DQN)

Mnih et al. (2013) introduced DQN, which approximates $Q(s, a; \theta)$ using a deep neural network with parameters θ. The network is trained to minimize the temporal-difference error:[MKS+13]

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)\right)^2\right] \tag{3.26}$$

where $\mathcal{D}$ is a replay buffer of stored transitions and θ^- are the parameters of a target network.

DQN introduces two key innovations to stabilize training:

Experience Replay. Transitions (s, a, r, s') are stored in a replay buffer $\mathcal{D}$ and sampled uniformly during training. This breaks temporal correlations in the data and enables more efficient reuse of experience.

Target Network. A separate target network with parameters θ^- is used to compute TD targets. These parameters are periodically updated from the main network ($\theta^- \leftarrow \theta$ every C steps), providing more stable learning targets.

Double Deep Q-Network (Double DQN)

Standard DQN tends to overestimate Q-values due to the max operator in the Bellman update.[vHGS15] **Van Hasselt et al. (2015)** propose Double DQN, which decouples action selection from action evaluation:

$$Y_t^{\text{DoubleDQN}} = r + \gamma Q(s', \arg\max_{a'} Q(s', a'; \theta); \theta^-) \tag{3.27}$$

The online network selects the best action, but the target network evaluates it. This significantly reduces overestimation bias and improves learning stability.

Dueling DQN

Wang et al. (2016) introduce a network architecture that separately estimates the state value $V(s)$ and the advantage of each action $A(s, a)$:[WSH+16]

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta, \alpha) \right) \quad (3.28)$$

where θ represents shared parameters, α are advantage stream parameters, and β are value stream parameters. This decomposition allows the network to learn which states are valuable independent of the action choice, accelerating learning.

Prioritized Experience Replay

Schaul et al. (2016) improve experience replay by sampling transitions non-uniformly based on their TD error.[SQAS16] Transitions with higher error $|\delta| = |r + \gamma \max_{a'} Q(s', a') - Q(s, a)|$ are sampled more frequently, as they provide more learning signal. The sampling probability is:

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha} \quad (3.29)$$

where $p_i = |\delta_i| + \epsilon$ and α controls the degree of prioritization.

Application to Combinatorial Optimization

DQN and its variants have been successfully applied to discrete optimization problems by formulating them as sequential decision processes [LZYZ23]. For constructive heuristics, each placement decision can be treated as an action, with the Q-network learning to evaluate the long-term quality of placing an item at a specific location. The discrete action space (selecting from candidate placements) is well-suited to Q-learning methods.

3.2.5 Transformer Networks

Transformers, introduced by **Vaswani et al. (2017)**, represent a paradigm shift in sequence modeling by replacing recurrence with self-attention mechanisms.[VSP+17] This architecture has become dominant in natural language processing and is increasingly applied to combinatorial optimization.

Self-Attention Mechanism

The core component of transformers is the scaled dot-product attention. Given query $\mathbf{Q}$, key $\mathbf{K}$, and value $\mathbf{V}$ matrices:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V} \tag{3.30}$$

where d_k is the dimension of the key vectors. The scaling factor $1/\sqrt{d_k}$ prevents dot products from growing too large.

For a sequence of inputs $\mathbf{X} = [\mathbf{x}_1, \ldots, \mathbf{x}_n]$, self-attention computes:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V \tag{3.31}$$

$$\mathbf{Z} = \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \tag{3.32}$$

where $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ are learnable projection matrices.

Multi-Head Attention

Transformers employ multiple attention heads operating in parallel, each learning different aspects of the relationships:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}_O \tag{3.33}$$

where $\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_Q^i, \mathbf{K}\mathbf{W}_K^i, \mathbf{V}\mathbf{W}_V^i)$ and $\mathbf{W}_O$ is an output projection matrix.

Transformer Architecture

A full transformer layer consists of:

1. Multi-head self-attention with residual connection and layer normalization:
$$\mathbf{Z} = \text{LayerNorm}(\mathbf{X} + \text{MultiHead}(\mathbf{X}, \mathbf{X}, \mathbf{X})) \tag{3.34}$$
2. Position-wise feedforward network:
$$\text{FFN}(\mathbf{z}) = \max(0, \mathbf{z}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2 \tag{3.35}$$
applied with residual connection:
$$\mathbf{Y} = \text{LayerNorm}(\mathbf{Z} + \text{FFN}(\mathbf{Z})) \tag{3.36}$$

Positional Encoding

Since attention is permutation-equivariant, position information must be injected explicitly. The original transformer uses sinusoidal positional encodings:

$$\text{PE}_{(pos,2i)} = \sin(pos/10000^{2i/d}) \tag{3.37}$$
$$\text{PE}_{(pos,2i+1)} = \cos(pos/10000^{2i/d}) \tag{3.38}$$

Alternatively, learned positional embeddings can be used.

Set Transformers

Lee et al. (2019) adapt transformers for set-structured inputs by introducing Induced Set Attention Blocks (ISAB) that reduce computational complexity from $O(n^2)$ to $O(nm)$ where $m \ll n$ is the number of inducing points.[LLK+19]

An ISAB computes attention through a bottleneck:

$$\mathbf{H} = \text{Attention}(\mathbf{I}, \mathbf{X}, \mathbf{X}) \tag{3.39}$$
$$\mathbf{Z} = \text{Attention}(\mathbf{X}, \mathbf{H}, \mathbf{H}) \tag{3.40}$$

where $\mathbf{I} \in \mathbb{R}^{m \times d}$ are learnable inducing points.

Set transformers are particularly relevant for combinatorial optimization because many problem instances are naturally represented as sets (e.g., sets of items, candidate placements, or graph nodes) where the order is irrelevant.

Application to Optimization

Transformers have been successfully applied to wide variety of continuous optimization tasks. Their ability to model long-range dependencies and attend to all elements simultaneously makes them effective for problems where global context is important. For packing problems, transformers can attend over items and candidate placements, learning complex relationships between geometric configurations.

In the context of bin packing, **Xiong et al. (2021)** use a Packing Transformer to model interactions between items and empty maximal spaces, demonstrating that attention mechanisms can capture spatial relationships relevant for placement decisions.[ZLZ+21]

3.3 Neurogenesis

Neurogenesis or neuroevolution is a practice of evolving the architecture and hyperparameters of given neural scoring models with a genetic algorithm. Classical neurogenesis like NEAT established that evolving topology alongside weights is effective.[LSX+20] Latter techniques such as Efficient Neural Architecture Search, shortly ENAS, focuses mostly on development of deep neural networks.[RMS+17]

Artificial neural network architecture design is a nontrivial and time-consuming task that often requires a high level of human expertise. Neurogenesis serves to automate the design of NN architectures and has proven to be successful in automatically finding NN architectures that outperform those manually designed by human experts.[BB24]

3.3.1 Principles

In principle, neural network design is viewed as a population-based search over both structure (topology/connection patterns) and parameters. Individuals encode candidate architectures and hyperparameters. Each is initiated, evaluated by an outcome-based fitness function. The target fitness is to be improved via actions of genetic algorithm of selection, crossover, mutation, and elitism.

3.3.2 Fitness design

Fitness is computed by outcome: each individual (architecture + hyperparameters) is instantiated, run on a batch of task instances, and mapped to a scalar for elitism. A simple scalarization is

$$\text{Fit} \;=\; -\,\text{TaskCost} \;-\; \lambda_{\text{viol}} \cdot \text{Violations} \;+\; \lambda_{Q} \cdot \text{Quality} \;-\; \lambda_{t} \cdot \text{Runtime},$$

with weights set to reflect priorities. Alternatively, multiple objectives can be kept in a Pareto set (non-dominated sorting).[EMH19] To reduce variance and cost, average fitness over small instance batches and use multi-fidelity evaluation (cheap proxies early; full fidelity for shortlisted elites).[WSS+23]

3.3.3 Search space and genome encoding

We encode a candidate network as a vector of genes. Compact search space keeps evaluation times manageable. Some of the examples include following:[WSS+23]

- **Depth/width:** number of layers (e.g., 2–6); hidden sizes (e.g., 64/128/256/512).
- **Activations & normalization:** {ReLU, GELU, SiLU} and {none, LayerNorm, BatchNorm}.
- **Skip/residuals:** on/off per stage.
- **Context aggregation (optional):** {none, local MLP pooling, small GNN over kNN}.
- **Feature subset:** bitmask/index selecting which engineered features are used.
- **Regularization/training:** dropout rate, weight decay, learning-rate schedule.

Gene	Example domain
Depth	$\{2, 3, 4, 5, 6\}$
Hidden width	$\{64, 128, 256, 512\}$
Activation	{ReLU, GELU, SiLU}
Normalization	{none, LayerNorm, BatchNorm}
Residuals	{off, on}
Context aggregation	{none, MLP pooling, small GNN}
Feature subset	index / bitmask over features
Dropout	[0, 0.3]
Weight decay	[0, 0.1]
LR schedule	{cosine, step, warmup}

Table 3.2: Genome (architecture & training) encoding for the scoring model.

3.3.4 Evolutionary operators

Evolutionary operators in neurogenesis follow standard genetic algorithm, specialized for neural networks by using a genome that encodes architecture and hyperparameters. The operators below are adapted to these NN-specific genomes.

Initialization. Sample genomes uniformly; optionally seed a few hand-designed baselines.

Selection. Tournament or rank-based selection; for distance vs. runtime trade-offs, use non-dominated sorting (NSGA-II style).

Crossover. One-point or uniform crossover for discrete genes; arithmetic/BLX-α blending for continuous ones.[TK01]

Mutation. Small edits - flip activation/normalization, ± 1 layer, resample feature subset, jitter dropout/weight decay, optionally self-adaptive mutation rates.[LSX+20]

Elitism, replacement. Keep the top E elites; fill the rest with best offspring; optionally add age/diversity pressure.

3.3.5 Extensions of GA

Latter works extend the classical genetic algorithm framework with several innovations:

Hyperparameter Co-Evolution. Winter et al. (2025) propose Neuvo NAS+, a system that evolves training hyperparameters alongside network architecture rather than fixing them[WT25]. This approach allows discovery of synergistic combinations where, for example, deeper networks are paired with lower learning rates. Their method optimizes learning rate, batch size, and discount factor (for reinforcement learning) as evolvable genes, demonstrating that task-specific co-optimization significantly outperforms approaches with fixed hyperparameters.

Adaptive Population Sizing. Prathiba et al. (2024) use adaptive population control where population size dynamically adjusts across generations based on the evolution phase[PLLP+24]:

- *Early phase* (exploration): Larger population (1.5× base) for diversity
- *Middle phase* (stable): Normal population size
- *Late phase* (exploitation): Smaller population (0.75× base) for refinement

This strategy adapts computational resources to search needs, reducing time to find good solutions.

Diversity-Driven Adaptation. Dendorfer et al. (2025) develop Population-Based Guiding, a framework that monitors population diversity and adaptively adjusts mutation rates[DK25]. Their approach incorporates architecture embeddings to guide mutations toward unexplored regions; when diversity drops below thresholds (measured via coefficient of variation), mutation rates are temporarily boosted to prevent premature convergence. The guided mutation strategy steers the search toward underexplored areas, achieving up to three times faster convergence compared to regularized evolution baselines.

3.4 Gaps in Literature

Despite progress in neural combinatorial optimization and reinforcement learning for bin packing, several critical gaps motivate this work:

3.4.1 Absence of Training Datasets

Unlike supervised learning domains (computer vision, NLP), no standardized datasets exist for 3D bin packing. Generating optimal solutions requires exponential-time exact solvers that do not scale[MPV98], making supervised approaches impractical. This necessitates reinforcement learning, which learns policies from interaction rather than labeled examples.

3.4.2 Architecture Design for Spatial Reasoning

Standard fully connected networks are poorly suited for 3D bin packing due to the domain's inherent spatial structure. Most neural approaches therefore incorporate specialized components for spatial reasoning[LZYZ23, XGP+24]: CNNs for heightmap processing, attention mechanisms for item relationships, or graph networks for geometric constraints. Key challenges include:

- Spatial relationships (heightmaps, contact surfaces, 3D geometry)

- Permutation invariance (item order independence)
- Variable-size action spaces (dynamic as bins fill)

While some works apply CNNs or attention mechanisms[XGP+24], no consensus exists on optimal architectures, and manual design requires extensive trial-and-error. **No prior work we are aware of applies automated neural architecture search to this domain.**

3.4.3 Neuroevolution for RL in Combinatorial Optimization

Most neural architecture search research targets supervised learning (image classification)[LSX+20, WSS+23]. The few works combining NAS with RL focus on control tasks (robotics, Atari)[LHT+24], not combinatorial problems. Additionally, existing approaches:

- Evolve architecture while fixing training hyperparameters
- Do not address RL-specific challenges (sparse rewards, credit assignment)

Gap: No work jointly evolves architecture and hyperparameters using genetic algorithms specifically for DQN-based combinatorial solvers.

3.4.4 Sparse Reward Challenge

Standard DQN literature addresses domains with frequent rewards (Atari games, robotic control)[MKS+13]. Bin packing exhibits **extremely sparse rewards**:

- No intermediate feedback during bin filling
- Reward only at episode termination
- Credit assignment across 50+ sequential decisions

While reward shaping could help, **most existing RL works for bin packing do not explicitly address sparsity**[LZYZ23].

3.4.5 Offline Bin Packing

The majority of bin packing literature—classical and neural—focuses on online variant where items arrive sequentially. However, we focus on potentially more complex and likely more universal(warehouse fulfillment, container loading) offline scenario. This enables lookahead, item reordering, and batch processing—yet neural RL for offline bin packing remains under-explored.

3.4.6 Contributions of This Work

This thesis addresses these gaps by:

1. Developing a DQN architecture incorporating spatial features (CNN) and relational reasoning (attention) for offline 3D bin packing
2. Applying modern, problem-specific neuroevolution to automatically discover effective architecture for non-standard network required for bin-packing
3. Designing reward shaping to mitigate sparse rewards
4. Evaluating on realistic benchmarks with multiple constraints

Chapter 4

Proposed method

stuff from research, make some graphs, callback to problem definition in ch2

Chapter 5

Implementation

Chapter 6

Experiments

Chapter 7

Conclusions

Appendix A

Bibliography

[ARCO22] Sara Ali, António Galrão Ramos, Maria Antónia Carravilla, and José Fernando Oliveira, *On-line three-dimensional packing problems: A review of off-line and on-line solution approaches*, Computers & Industrial Engineering **168** (2022), 108122.

[BB24] Reinhard Booysen and Anna Sergeevna Bosman, *Multi-objective evolutionary neural architecture search for recurrent neural networks*, Neural Processing Letters **56** (2024), no. 4.

[BTC23] Guillem Bonet Filella, Alessio Trivella, and Francesco Corman, *Modeling soft unloading constraints in the multi-drop container loading problem*, European Journal of Operational Research **308** (2023), no. 1, 336–352.

[CPT08] Teodor Gabriel Crainic, Guido Perboli, and Roberto Tadei, *Extreme point-based heuristics for three-dimensional bin packing*, INFORMS J. Comput. **20** (2008), 368–384.

[CvMG+14] Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio, *Learning phrase representations using rnn encoder-decoder for statistical machine translation*, 2014.

[DK25] Stefan Dendorfer and Andreas M. Kist, *Population-based guiding for evolutionary neural architecture search*, Scientific Reports **15** (2025), no. 1, 38996.

[dNAJ21] Oliviana Xavier do Nascimento, Thiago Alves de Queiroz, and Leonardo Junqueira, *Practical constraints in the container load-*

ing problem: Comprehensive formulations and exact algorithm, Computers & Operations Research **128** (2021), 105186.

[DSC22] Shiv Ram Dubey, Satish Kumar Singh, and Bidyut Baran Chaudhuri, *Activation functions in deep learning: A comprehensive survey and benchmark*, 2022.

[DXSZ10] Shifei Ding, Li Xu, Chunyang Su, and Hong Zhu, *Using genetic algorithms to optimize artificial neural networks.*, JCIT **5** (2010), 54–62.

[Elm90] Jeffrey L. Elman, *Finding structure in time*, Cognitive Science **14** (1990), no. 2, 179–211.

[EMH19] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter, *Neural architecture search: A survey*, Journal of Machine Learning Research **20** (2019), no. 55, 1–21, Editor: Sebastian Nowozin.

[FB10] Tobias Fanslau and Andreas Bortfeldt, *A tree search algorithm for solving the container loading problem*, INFORMS Journal on Computing **22** (2010), no. 2, 222–235.

[GOGL16] A. Galrão Ramos, José F. Oliveira, José F. Gonçalves, and Manuel P. Lopes, *A container loading algorithm with static mechanical equilibrium stability constraints*, Transportation Research Part B: Methodological **91** (2016), 565–581.

[GR13] José Fernando Gonçalves and Mauricio G.C. Resende, *A biased random key genetic algorithm for 2d and 3d bin packing problems*, International Journal of Production Economics **145** (2013), no. 2, 500–510.

[GSC00] Felix A. Gers, Jürgen A. Schmidhuber, and Fred A. Cummins, *Learning to forget: Continual prediction with lstm*, Neural Comput. **12** (2000), no. 10, 2451–2471.

[GSR+17] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl, *Neural message passing for quantum chemistry*, 2017.

[GTMP22] Mikele Gajda, Alessio Trivella, Renata Mansini, and David Pisinger, *An optimization approach for a complex real-life container loading problem*, Omega **107** (2022), 102559.

[HHW24] Katrin Heßler, Timo Hintsch, and Lukas Wienkamp, *A fast optimization approach for a complex real-life 3d multiple bin size bin packing problem*, 2024.

[KSH12] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton, *Imagenet classification with deep convolutional neural networks*, Proceedings of the 26th International Conference on Neural

Information Processing Systems - Volume 1 (Red Hook, NY, USA), NIPS'12, Curran Associates Inc., 2012, p. 1097–1105.

[KW17] Thomas N. Kipf and Max Welling, *Semi-supervised classification with graph convolutional networks*, 2017.

[LBBH98] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, *Gradient-based learning applied to document recognition*, Proceedings of the IEEE **86** (1998), no. 11, 2278–2324.

[LHT+24] Pengyi Li, Jianye Hao, Hongyao Tang, Xian Fu, Yan Zheng, and Ke Tang, *Bridging evolutionary algorithms and reinforcement learning: A comprehensive survey on hybrid algorithms*, 2024.

[LLK+19] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam R. Kosiorek, Seungjin Choi, and Yee Whye Teh, *Set transformer: A framework for attention-based permutation-invariant neural networks*, 2019.

[LSX+20] Yuqiao Liu, Yanan Sun, Bing Xue, Mengjie Zhang, and Gary G. Yen, *A survey on evolutionary neural architecture search*, CoRR **abs/2008.10937** (2020).

[LZYZ23] Huwei Liu, Li Zhou, Jianglong Yang, and Junhui Zhao, *The 3d bin packing problem for multiple boxes and irregular items based on deep q-network*, Applied Intelligence **53** (2023), 1–28.

[LZZ14] Xueping Li, Zhaoxia Zhao, and Kaike Zhang, *A genetic algorithm for the three-dimensional bin packing problem with heterogeneous bins*, IIE Annual Conference and Expo 2014 (2014).

[MKS+13] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller, *Playing atari with deep reinforcement learning*, 2013.

[MPV98] Silvano Martello, David Pisinger, and Daniele Vigo, *The three-dimensional bin packing problem*, Operations Research **48** (1998).

[MSIB20] Nina Mazyavkina, Sergey Sviridov, Sergei Ivanov, and Evgeny Burnaev, *Reinforcement learning for combinatorial optimization: A survey*, 2020.

[MZF+25] Ying Ma, Yu Zhou, Qiwei Fang, Shengwei Xia, and Wei Chen, *A three-dimensional container loading algorithm for solving logistics packing problem*, EURO Journal on Transportation and Logistics **14** (2025), 100167.

[PAVTO08] F. Parreòo, R. Alvarez-Valdes, J. M. Tamarit, and J. F. Oliveira, *A maximal-space algorithm for the container loading problem*, INFORMS J. on Computing **20** (2008), no. 3, 412–422.

[PLLP+24] L. Prathiba, S. Lavanya, D. Lakshmi Padmaja, B. Gokulavasan, and K. Nethra, *Evolving neural architectures: A genetic algorithm approach to deep learning optimization*, Proceedings of the 3rd International Conference on Optimization Techniques in the Field of Engineering (ICOFE-2024), November 2024, Available at SSRN: `https://ssrn.com/abstract=5115996`.

[RMS+17] Esteban Real, Sherry Moore, Andrew Selle, Saurabh Saxena, Yutaka Leon Suematsu, Jie Tan, Quoc Le, and Alex Kurakin, *Large-scale evolution of image classifiers*, 2017.

[RSO18] António G. Ramos, Elsa Silva, and José F. Oliveira, *A new load balance methodology for container loading problem in road transportation*, European Journal of Operational Research **266** (2018), no. 3, 1140–1152.

[SB18] Richard S. Sutton and Andrew G. Barto, *Reinforcement learning: An introduction*, second ed., The MIT Press, 2018.

[SGT+09] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini, *The graph neural network model*, IEEE Transactions on Neural Networks **20** (2009), 61–80.

[SQAS16] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver, *Prioritized experience replay*, 2016.

[TK01] M. Takahashi and H. Kita, *A crossover operator using independent component analysis for real-coded genetic algorithms*, Proceedings of the 2001 Congress on Evolutionary Computation (IEEE Cat. No.01TH8546), vol. 1, 2001, pp. 643–649 vol. 1.

[VCC+18] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio, *Graph attention networks*, 2018.

[vHGS15] Hado van Hasselt, Arthur Guez, and David Silver, *Deep reinforcement learning with double q-learning*, 2015.

[VSP+17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin, *Attention is all you need*, CoRR **abs/1706.03762** (2017).

[WFH+23] Wenjie Wu, Changjun Fan, Jincai Huang, Zhong Liu, and Junchi Yan, *Machine learning for the multi-dimensional bin packing problem: Literature review and empirical evaluation*, 2023.

[WHS07] Gerhard Wäscher, Heike Haußner, and Holger Schumann, *An improved typology of cutting and packing problems*, European Journal of Operational Research **183** (2007), no. 3, 1109–1130.

[WSH+16] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, and Nando de Freitas, *Dueling network architectures for deep reinforcement learning*, 2016.

[WSS+23] Colin White, Mahmoud Safari, Rhea Sukthanker, Binxin Ru, Thomas Elsken, Arber Zela, Debadeepta Dey, and Frank Hutter, *Neural architecture search: Insights from 1000 papers*, 2023.

[WT25] Benjamin David Winter and William John Teahan, *Evaluating a novel neuroevolution and neural architecture search system*, 2025.

[XGP+24] Heng Xiong, Changrong Guo, Jian Peng, Kai Ding, Wenjie Chen, Xuchong Qiu, Long Bai, and Jianfeng Xu, *Gopt: Generalizable online 3d bin packing via transformer-based deep reinforcement learning*, IEEE Robotics and Automation Letters **9** (2024), no. 11, 10335–10342.

[ZLZ+21] Qianwen Zhu, Xihan Li, Zihan Zhang, Zhixing Luo, Xialiang Tong, Mingxuan Yuan, and Jia Zeng, *Learning to pack: A data-driven tree search algorithm for large-scale 3d bin packing problem*, 10 2021, pp. 4393–4402.

[ZZOL12] Wenbin Zhu, Zhaoyi Zhang, Wee-Chong Oon, and Andrew Lim, *Space defragmentation for packing problems*, European Journal of Operational Research **222** (2012), no. 3, 452–463.

Katedra: matematiky Akademický rok: 2008/2009

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

Pro:	Tomáš Hejda
Obor:	Matematické inženýrství
Zaměření:	Matematické modelování
Název práce:	Spřátelené morfismy na sturmovských slovech / Amicable Morphisms on Sturmian Words

Osnova:

1. Seznamte se se základními pojmy a větami z teorie symbolických dynamických systémů.
2. Udělejte rešerši poznatků o sturmovských slovech: přehled ekvivalentních definic sturmovských slov, popis morfismů zachovávajících sturmovská slova, popis standardních párů slov.
3. Zkoumejte vlastnosti párů spřátelených sturmovských morfismů, pokuste se popsat jejich generování a počty v závislosti na tvaru jejich matice.

Doporučená literatura:

1. M. Lothair, Algebraic Combinatorics on Words, Encyclopedia of Math. and its Applic., Cambridge University Press, 1990
2. J. Berstel, Sturmian and episturmian words (a survey of some recent result results), in: S. Bozapalidis, G. Rahonis (eds), Conference on Algebraic Informatics, Thessaloniki, Lecture Notes Comput. Sci. 4728 (2007), 23-47.
3. P. Ambrož, Z. Masáková, E. Pelantová, Morphisms fixing a 3iet words, preprint DI (2008)

Vedoucí bakalářské práce:	Prof. Ing. Edita Pelantová, CSc.
Adresa pracoviště:	Fakulta Jaderná a fyzikálně inženýrská Trojanova 13 / 106 Praha 2

Konzultant:

Datum zadání bakalářské práce:	15.10.2008
Termín odevzdání bakalářské práce:	**7.7.2009**

V Praze dne 17.3.2009

....

Vedoucí katedry Děkan